Plan de tests hiérarchisé

Vérification complète du code MicroPython du capteur porté par le joueur (BNO055 + GPS, modes foot et rugby), du câblage jusqu'à la validation du cahier des charges.

Principe

Les tests sont rangés en neuf niveaux, du plus élémentaire au plus global. Chaque niveau suppose que les tests **bloquants** des niveaux dont il dépend sont réussis : sans cela, une erreur observée plus haut ne peut pas être attribuée à la bonne cause (câblage, capteur, calcul ou réglage).

Prio	Signification
B	**Bloquant** : tant qu'il échoue, ne pas passer aux niveaux qui en dépendent.
I	**Important** : à réussir avant la validation finale (niveau 8).
C	**Complémentaire** : utile, mais sans effet sur les modes foot et rugby.

Vue d'ensemble

Niv.	Objet	Lieu	Matériel	Prérequis
0	Préparation du Pico et des fichiers	Bureau	PC, câble USB, mpremote ou Thonny	—
1	Câblage et bus (I2C, série)	Bureau	Pico, BNO055, GPS	0
2	Capteur BNO055 seul	Bureau	Table plane	1
3	GPS seul	Extérieur	Ciel dégagé, décamètre	1
4	Calculs hors ligne (sans mouvement)	Bureau	Pico seul	0
5	Mode rugby (capteur au col du maillot)	Terrain	Maillot, décamètre, chrono, vidéo	2, 4
6	Mode foot (capteur au mollet)	Terrain	Protège-tibia, ballons, radar, vidéo	2, 4
7	Assistance GPS (secours, étalonnage, position)	Terrain	+ vélo ou chariot	3, 5, 6
8	Validation du cahier des charges	Terrain	Piste 400 m, radar, chronométrage, vidéo	5, 6, 7

Comment lancer les tests (Thonny). Les commandes d'une ligne se tapent dans la console (en bas). Les scripts S1 à S8 se collent dans l'éditeur et se lancent avec le bouton **Run** : Thonny redémarre alors le Pico (« MPY: soft reboot »), c'est pourquoi chaque script contient ses propres import. Les valeurs attendues des scripts S6 à S8 ont été vérifiées en exécutant le code.

Critères chiffrés. Le cahier des charges ne fixe pas de précision chiffrée. Les tolérances proposées (±2 %, ±5 %…) sont des objectifs raisonnables pour ce type de capteur, à ajuster selon les besoins de l'équipe.

Niveau 0 — Préparation

Objectif : disposer d'un Pico 2 W avec MicroPython et tous les fichiers du projet.

Lieu : bureau | **Matériel :** PC, câble USB | **Prérequis :** aucun

ID	Prio	Test	Procédure	Critère de réussite	OK
T0.1	**B**	**Firmware MicroPython**	Flasher le firmware MicroPython pour Raspberry Pi Pico 2 W (micropython.org). Dans le REPL : `import sys; print(sys.implementation)`	La version de MicroPython s'affiche. *Si échec : brancher le Pico en maintenant BOOTSEL, recopier le fichier .uf2.*	☐
T0.2	**B**	**Copie des fichiers**	Thonny : **Affichage → Fichiers** ; en haut (Cet ordinateur), sélectionner les 8 fichiers (dont journal.py) ; **clic droit → Téléverser vers /**.	Les 8 fichiers apparaissent dans le panneau du bas (Raspberry Pi Pico), à la racine. *Si échec : bno055_base.py provient du dépôt micropython-bno055 de Peter Hinch.*	☐
T0.3	**B**	**Imports**	`import capteur_bno055, capteur_gps, deplacement, journal, mode_foot, mode_rugby`	Aucune erreur. *Si échec : ImportError : fichier manquant. SyntaxError : firmware trop ancien, le mettre à jour.*	☐
T0.4	**I**	**Mémoire libre**	`import gc; gc.collect(); print(gc.mem_free())` après T0.3.	Noter la valeur : elle limite la durée de enregistrer() (16 octets par échantillon).	☐

Niveau 1 — Câblage et bus

Objectif : vérifier que le Pico dialogue avec les deux capteurs.

Lieu : bureau | **Matériel :** Pico, BNO055 (I2C0 : GP4/GP5), GPS (UART0 : GP0/GP1) | **Prérequis :** niveau 0

ID	Prio	Test	Procédure	Critère de réussite	OK
T1.1	**B**	**Bus I2C**	Script S1, ligne T1.1.	`['0x28']` (ou `'0x29'` si ADR relié à 3,3 V). *Si échec : liste vide : SDA/SCL inversés, alimentation, masse commune. OSError : `creer_bus_i2c(frequence_hz=100000)`.*	☐
T1.2	**B**	**Détection du BNO055**	Script S1, ligne T1.2.	Pas d'exception ; `c.mode_lu()` renvoie 12 (mode NDOF après reset). *Si échec : RuntimeError « introuvable » : voir T1.1. OSError [Errno 5] EIO : faux contact (souder les barrettes, fils courts). « bad chip id » : mauvais composant ou adresse.*	☐
T1.3	**B**	**Réception du GPS**	Script S1, ligne T1.3 (GPS alimenté depuis au moins 2 s).	Texte contenant des trames `$GPRMC` / `$GPGGA` (ou préfixe `$GN`). *Si échec : None : TX/RX inversés ou GPS non alimenté. Caractères illisibles : débit différent de 9600 bauds.*	☐
T1.4	**I**	**Coexistence**	Refaire T1.1 et T1.3 avec les deux capteurs branchés en même temps.	Mêmes résultats que séparément.	☐

Script S1 — bus et détection

```
from capteur_bno055 import creer_bus_i2c, CapteurBNO055
i2c = creer_bus_i2c()
print([hex(a) for a in i2c.scan()])     # T1.1 : ['0x28']
c = CapteurBNO055(i2c)
print(c.mode_lu())                       # T1.2 : 12

from capteur_gps import creer_uart_gps
import time
u = creer_uart_gps(); time.sleep(2)
print(u.read())                          # T1.3 : b'$GPRMC,...'
```

Niveau 2 — Capteur BNO055 seul

Objectif : vérifier les réglages, les mesures au repos, les axes, le gyroscope et le cadencement.

Lieu : bureau | **Matériel :** table plane | **Prérequis :** niveau 1

ID	Prio	Test	Procédure	Critère de réussite	OK
T2.1	**B**	**Mode foot**	`c.passer_en_mode_foot(); print(c.mode_lu())`	Pas d'exception (la méthode relit et vérifie les réglages) ; affiche 5 (ACCGYRO). *Si échec : RuntimeError « Configuration refusee » : refaire T1.2, vérifier l'alimentation.*	☐
T2.2	**B**	**Mode rugby**	`c.passer_en_mode_rugby(); print(c.mode_lu())`	Pas d'exception ; affiche 5.	☐
T2.3	**B**	**Accélération au repos**	Capteur posé à plat, script S2, ligne T2.3.	Norme comprise entre 9,4 et 10,2 m/s². *Si échec : valeur proche de 0 : registres mal lus ; valeur ×10 ou ÷10 : unités modifiées.*	☐
T2.4	**B**	**Sens des axes**	Poser le boîtier sur chacune de ses 6 faces ; lire `c.lire_acceleration()`.	L'axe vertical lit +9,8 ou −9,8 (±0,5) ; les deux autres ≈ 0. Noter le repère des axes sur le boîtier (utile pour la fixation).	☐
T2.5	**B**	**Biais du gyroscope**	Immobile, script S2, lignes T2.5.	Chaque composante du biais |b| < 3 °/s ; ensuite `lire_gyroscope()` de l'ordre de ±1 °/s au plus par axe (bruit d'une lecture isolée). *Si échec : biais élevé : refaire au repos complet, loin des vibrations.*	☐
T2.6	**B**	**Rejet du mouvement**	Bouger le capteur pendant `c.estimer_biais_gyro()`.	RuntimeError « Mouvement detecte... ».	☐
T2.7	**I**	**Échelle du gyroscope**	Script S3 : capteur à plat, lui faire faire exactement un tour (repère sur la table) en moins de 10 s.	Angle affiché ±360° (±10°). *Si échec : erreur proportionnelle : vérifier T2.5 puis le réglage du mode.*	☐
T2.8	**B**	**Fréquence maximale**	Script S2, ligne T2.8 (après passer_en_mode_foot).	≥ 180 Hz (pour 125 Hz en mode foot, à 70 % au plus de la capacité). Valeur mesurée sur le montage : ≈ 210 Hz, limite propre au BNO055 (≈ 0,8 ms + 0,22 ms par octet lu), identique à 100 et 400 kHz. *Si échec : réduire FREQUENCE_FOOT_HZ (ex. 100) dans mode_foot.py et refaire T6.2.*	☐
T2.9	**B**	**Cadencement**	Script S4 (10 s en mode foot puis 10 s en mode rugby).	foot : ≈ 1250 échantillons ; rugby : ≈ 1000 ; nb_manques = 0.	☐

ID	Prio	Test	Procédure	Critère de réussite	OK
T2.10	I	**Enregistrement CSV**	`d, n = c.enregistrer(5, 125); from capteur_bno055 import sauvegarder_csv; sauvegarder_csv(d, n, "t.csv", c.biais_gyro)` puis le récupérer (Thonny : clic droit sur t.csv → Télécharger). Essayer aussi `c.enregistrer(600)`.	n ≈ 625, CSV de 7 colonnes lisible ; `enregistrer(600)` lève MemoryError avec un message clair.	☐
T2.11	C	**Température**	`print(c.lire_temperature())`	Température ambiante ±5 °C.	☐
T2.12	C	**Fusion et calibration**	`from capteur_bno055 import demo_orientation; demo_orientation(60)` ; tourner le capteur (6 faces, puis des « 8 »). Relancer.	Calibration OK puis calib_bno055.bin créé ; au 2e lancement « Calibration rechargee ». Angles stables à ±1° immobile.	☐
T2.13	C	**Garde-fous de mode**	En mode foot : `c.lire_euler()`	RuntimeError explicite (aucune valeur fausse renvoyée).	☐

Script S2 — réglages et repos

```
from capteur_bno055 import creer_bus_i2c, CapteurBNO055, norme3
c = CapteurBNO055(creer_bus_i2c())
c.passer_en_mode_foot(); print(c.mode_lu())       # T2.1 : 5
a = c.lire_acc_gyro_rapide()
print(norme3(a[0], a[1], a[2]))                     # T2.3 : 9,4 à 10,2
print(c.estimer_biais_gyro())                       # T2.5 : |b| < 3 deg/s
print(c.lire_gyroscope())                           # T2.5 : ~ +/-1 deg/s au plus
print(c.mesurer_frequence_max())                    # T2.8 : >= 180 Hz
```

Script S3 — échelle du gyroscope (T2.7)

```
from capteur_bno055 import creer_bus_i2c, CapteurBNO055
c = CapteurBNO055(creer_bus_i2c())
c.passer_en_mode_rugby(); c.estimer_biais_gyro()
angle = 0.0
t_prec = None
for t, ax, ay, az, gx, gy, gz in c.echantillons(100, 10):
    if t_prec is not None:
        angle += gz * (t - t_prec) / 1000000
    t_prec = t
print(angle)               # un tour à plat (axe z vertical) : +/-360
```

Script S4 — cadencement (T2.9)

```
from capteur_bno055 import creer_bus_i2c, CapteurBNO055
c = CapteurBNO055(creer_bus_i2c())
for mode, f in (("foot", 125), ("rugby", 100)):
    getattr(c, "passer_en_mode_" + mode)()
    n = 0
    for e in c.echantillons(f, 10):
        n += 1
    print(mode, n, c.nb_manques)     # environ 10 x f, manques = 0
```

Niveau 3 — GPS seul

Objectif : vérifier la configuration, le débit, la qualité du signal, la vitesse, la position et le cap.

Lieu : extérieur, ciel dégagé | **Matériel :** GPS, décamètre, boussole (téléphone) | **Prérequis :** niveau 1

ID	Prio	Test	Procédure	Critère de réussite	OK
T3.1	B	**Configuration u-blox**	Script S5, ligne T3.1.	Toutes les valeurs du dictionnaire valent True. *Si échec : None partout : fil Pico TX (GP0) → RX du GPS absent. False : commande refusée (autre modèle, ou fréquence trop élevée).*	☐

ID	Prio	Test	Procédure	Critère de réussite	OK
T3.2	**B**	**Débit à 5 Hz**	Script S5, fin (10 s).	45 à 50 positions RMC décodées ; `nb_erreurs` ≤ 2. *Si échec : ≈ 10 : configuration non appliquée (T3.1). Erreurs nombreuses : parasites sur la liaison.*	☐
T3.3	**B**	**Premier fix**	`from capteur_gps import demo_gps; demo_gps(300)`	« fix oui » avec sat ≥ 5 et hdop ≤ 2,5. Noter le délai (premier démarrage à froid : souvent de l'ordre de la minute, parfois plus). *Si échec : s'éloigner des bâtiments ; antenne céramique tournée vers le ciel.*	☐
T3.4	**B**	**Vitesse à l'arrêt**	demo_gps, immobile 2 min.	Vitesse affichée < 1 km/h la plupart du temps.	☐
T3.5	**I**	**Stabilité de position**	Immobile 5 min.	La position locale reste dans un cercle d'environ 5 m.	☐
T3.6	**I**	**Distance**	Marcher 50 m mesurés au décamètre, en ligne droite.	$\sqrt{(x^2 + y^2)} = 50 \pm 5$ m à l'arrivée.	☐
T3.7	**I**	**Cap**	Marcher vers le nord (boussole) ; afficher `g.cap_deg`.	≈ 0° (±20°) quand la vitesse dépasse 1 m/s.	☐
T3.8	**C**	**Autre modèle de GPS**	En cas de changement de récepteur : refaire T1.3, T3.2 à T3.4 ; pour un modèle non u-blox, `configuration_ublox=False`.	Mêmes critères (fréquence par défaut du modèle).	☐

Script S5 — configuration et débit du GPS

```
from capteur_gps import creer_uart_gps, configurer_ublox, RecepteurGPS
import time
u = creer_uart_gps()
print(configurer_ublox(u, 5))           # T3.1 : toutes les valeurs True
g = RecepteurGPS(u)
n = 0
t0 = time.ticks_ms()
while time.ticks_diff(time.ticks_ms(), t0) < 10000:
    if g.mettre_a_jour():
        n += 1
    time.sleep_ms(5)
print(n, g.nb_erreurs)                  # T3.2 : 45 à 50, erreurs <= 2
```

Niveau 4 — Calculs hors ligne

Objectif : vérifier les calculs sur des données connues, indépendamment des capteurs.

Lieu : bureau | **Matériel :** Pico seul (capteurs facultatifs) | **Prérequis :** niveau 0

ID	Prio	Test	Procédure	Critère de réussite	OK
T4.1	**B**	**Décodage NMEA**	Script S6, ligne T4.1 (trames de référence).	`True 48.1173 11.51667 11.5236 84.4 8 0.9 True` (aux arrondis près).	☐
T4.2	**B**	**Somme de contrôle**	Script S6, ligne T4.2 (trame altérée).	`False 1` : trame rejetée et comptée en erreur.	☐
T4.3	**B**	**Trame UBX**	Script S6, ligne T4.3 (commande 5 Hz).	`b'b56206080600c80001000100de6a'`	☐
T4.4	**B**	**Indicateurs sur profil connu**	Script S7 (profil : arrêt, 0→3 m/s en 1 s, 3 m/s pendant 20 s, 3→7 m/s en 1,5 s, 7 m/s pendant 3 s, 7→0 m/s en 2,5 s, arrêt).	distance 0,099 km ; activité 55,6 % ; sprint max 25,2 km/h ; course 0:27 ; sprints 1 ; accélérations 2 ; décélérations 1 ; durée 48 s.	☐
T4.5	**B**	**Modèle rugby (Weinberg)**	Script S8, lignes T4.5.	`course sprint marche course` ; `1.3416` ; `0.6667`.	☐
T4.6	**B**	**Modèle foot (foulée)**	Script S8, ligne T4.6.	`course 3.0905` (m).	☐

ID	Prio	Test	Procédure	Critère de réussite	OK
T4.7	I	**Fichiers de modèle**	Script S8, ligne T4.7.	`True False` : un modèle foot ne se charge pas dans le mode rugby.	☐
T4.8	I	**Utilitaires**	Script S8, ligne T4.8.	`-2.2832 12:34 62.83`	☐
T4.9	C	**Paramètres invalides**	Script S8, fin.	Trois lignes « OK : … » (ValueError levées).	☐
T4.10	B	**Journaux et numérotation**	Script S9.	numeros croissants ; etat CSV de 4 lignes (entête comprise) ; journal d'événements de 1 événement et 375 échantillons ; conversion CSV de 375 lignes. *Si échec : journal inactif : lire son champ « erreur » (mémoire pleine, écriture impossible).*	☐
T4.11	B	**Tolérance aux erreurs de lecture**	Script S10 (capteur simulé en panne).	Environ 390 échantillons produits et nb_erreurs = 10 (30 appels en échec, 3 relances par échantillon) : l'acquisition a continué. OSError seulement après 50 échecs d'affilée.	☐
T4.12	I	**Espace disponible**	`import journal; print(journal.espace_libre_octets())`	Noter la valeur : à 2,4 ko/min de journal d'état et 6 ko par frappe, elle borne la durée des séances enregistrables. *Si échec : sous ~200 ko : effacer les anciens fichiers de séance.*	☐

Script S6 — GPS sans récepteur (T4.1 à T4.3)

```
import binascii, capteur_gps as cg
class FauxUART:
    def __init__(self, d): self.d = d
    def any(self): return len(self.d)
    def read(self, n):
        r = self.d[:n]; self.d = self.d[n:]; return r
RMC = b"$GPRMC,123519,A,4807.038,N,01131.000,E,022.4,084.4,230394,003.1,W*6A\r\n"
GGA = b"$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47\r\n"
g = cg.RecepteurGPS(FauxUART(RMC + GGA))
print(g.mettre_a_jour(), g.latitude_deg(), g.longitude_deg(), g.vitesse_ms,
      g.cap_deg, g.nb_satellites, g.hdop, g.est_exploitable())            # T4.1
g2 = cg.RecepteurGPS(FauxUART(RMC.replace(b"*6A", b"*00")))
print(g2.mettre_a_jour(), g2.nb_erreurs)                                  # T4.2
print(binascii.hexlify(cg._trame_ubx(0x06, 0x08, bytes((0xC8, 0, 1, 0, 1, 0)))))   # T4.3
```

Script S7 — indicateurs sur un profil de vitesse connu (T4.4)

```
from deplacement import IndicateursSeance
ind = IndicateursSeance()
t = 0
def rampe(v0, v1, d):
    global t
    n = int(d * 100)
    for i in range(n):
        v = v0 + (v1 - v0) * i / n
        ind.mettre_a_jour(t, v, v * 0.01)
        t += 10000
for p in ((0, 0, 10), (0, 3, 1), (3, 3, 20), (3, 7, 1.5), (7, 7, 3), (7, 0, 2.5), (0, 0, 10)):
    rampe(*p)
print(ind.resume())
```

Script S8 — modèles et utilitaires (T4.5 à T4.9)

```
from mode_rugby import ModeleWeinberg, SeanceRugby
from mode_foot import ModeleFoulee, DetecteurFrappe
from deplacement import angle_pi, format_min_s, IndicateursSeance
m = ModeleWeinberg()
print(m.allure(20, 0.35), m.allure(20, 0.25), m.allure(5, 0.6), m.allure(20, None))  # T4.5
print(m.distance_cycle(25, 0.35))                                                    # T4.5
print(m.etalonner(100, {"marche": 0, "course": 90, "sprint": 0}, "course"))         # T4.5
f = ModeleFoulee()
print(f.allure(800, 0.714), f.distance_cycle(800, 0.714))                            # T4.6
f.sauvegarder("t.json")
print(ModeleFoulee().charger("t.json"), ModeleWeinberg().charger("t.json"))         # T4.7
print(angle_pi(4), format_min_s(754), DetecteurFrappe().vitesse_max_mesurable_kmh())  # T4.8
for essai in (lambda: IndicateursSeance(facteur_sprint=1.0),
              lambda: SeanceRugby(priorite="x"),
              lambda: DetecteurFrappe(seuil_fin_dps=500)):
    try:
        essai(); print("ERREUR : pas d'exception")
    except ValueError as e:
        print("OK :", e)                                                            # T4.9
```

Script S9 — journaux et numérotation (T4.10)

```
import journal
n1 = journal.prochain_numero("test")
n2 = journal.prochain_numero("test")
print(n1, n2)                                  # deux entiers consécutifs
je = journal.JournalEtat(journal.nom_unique("etat_test", "csv", n2), periode_s=1.0)
jv = journal.JournalEvenements(journal.nom_unique("ev_test", "bin", n2), frequence_hz=125)
for i in range(500):                           # 4 s a 125 Hz
    t = i * 8000
    je.mettre_a_jour(t, {"vitesse_kmh": 10, "source": "imu"})
    jv.ajouter(t, 0.0, 0.0, 9.81, 100.0, 0.0, 0.0)
    if i == 250:
        jv.declencher()
je.fermer(); jv.fermer()
print(je.resume())                             # lignes : 4, actif : True
print(jv.resume())                             # evenements : 1, 375 echantillons
print(journal.convertir_csv(journal.nom_unique("ev_test", "bin", n2),
                            "ev_test.csv", 375))        # 375
```

Script S10 — tolérance aux erreurs de lecture (T4.11)

```
import struct
from capteur_bno055 import CapteurBNO055, ACCGYRO_MODE
class I2CIntermittent:    # echoue sur 30 appels : _lire_brut relance 3 fois,
    def __init__(self): self.n = 0        # soit 30/3 = 10 echantillons perdus
    def readfrom_mem_into(self, a, r, buf):
        self.n += 1
        if 100 <= self.n < 130:
            raise OSError(5, "EIO")
        struct.pack_into("<9h", buf, 0, 0, 0, 980, 0, 0, 0, 0, 0, 0)
c = CapteurBNO055.__new__(CapteurBNO055)
c.i2c = I2CIntermittent(); c.adresse = 0x28; c._buf = bytearray(18)
c.mode_courant = ACCGYRO_MODE; c.biais_gyro = (0, 0, 0)
c.nb_echantillons = c.nb_manques = c.nb_erreurs = 0
n = 0
for _ in c.echantillons(200, 2):
    n += 1
print(n, c.nb_erreurs)              # environ 390 et 10 : l'acquisition a continue
```

Niveau 5 — Mode rugby — capteur au col du maillot

Objectif : valider la détection des pas, l'étalonnage, la distance, la vitesse, le cap et les indicateurs, sans GPS.

Lieu : terrain | **Matériel :** maillot avec boîtier plaqué, décamètre, chronomètre, vidéo | **Prérequis :** niveaux 2 et 4

> Lancer : `import mode_rugby`, puis `mode_rugby.demo_rugby(durée_s)` (l'état s'affiche chaque seconde) ou `mode_rugby.demo_etalonnage(allure, distance_m)`. Rester immobile et debout pendant la mesure du biais (1 s) au démarrage.

ID	Prio	Test	Procédure	Critère de réussite	OK
T5.1	**B**	**Démarrage**	`demo_rugby(30)` debout immobile.	Aucune erreur ; vitesse 0 ; nb_pas ≤ 1 ; source « imu ».	☐
T5.2	**B**	**Verticale**	Immobile : se pencher de 30° en avant, puis se redresser.	Vitesse reste 0 ; nb_pas n'augmente pas.	☐
T5.3	**B**	**Pas en marche**	Marcher 40 pas comptés à voix haute.	nb_pas = 40 ± 1. *Si échec : ajuster seuil_min / fraction_seuil de creer_detecteur_pas().*	☐
T5.4	**B**	**Pas en course**	Courir 60 pas (compter les appuis d'un pied × 2).	nb_pas = 60 ± 2.	☐
T5.5	**B**	**Pas en sprint**	Sprint de 30 m filmé ; compter les appuis au ralenti.	Écart ≤ 2 pas. *Si échec : pas manqués : baisser intervalle_min_s (0,18 s) avec prudence.*	☐
T5.6	**I**	**Faux pas**	30 s de sauts sur place, rotations du tronc, assis-debout.	Noter les pas parasites (objectif ≤ 5). Les sauts sur place sont une limite connue.	☐
T5.7	**I**	**Rotation**	Tourner sur place de 90°, puis de 180°.	cap_deg varie de 90 ± 10°, puis de 180 ± 15°.	☐
T5.8	**B**	**Étalonnage**	`demo_etalonnage("marche", 50)`, puis `("course", 100)`, puis `("sprint", 40)` (départ lancé).	Trois K affichés, entre 0,1 et 2,0 sans être en butée ; modele_rugby.json créé. *Si échec : K en butée : distance mal mesurée ou allure non tenue ; recommencer.*	☐
T5.9	**B**	**Distance**	3 × 100 m en course régulière, sans GPS.	distance_km = 0,100 ± 0,005 à chaque essai.	☐
T5.10	**B**	**Vitesse moyenne**	100 m chronométrés (temps t en s).	vitesse_moy_course_kmh = 360 / t ± 5 %.	☐
T5.11	**I**	**Position à l'estime**	Aller-retour de 50 m.	Retour à moins de 10 m de l'origine (x_m, y_m).	☐
T5.12	**I**	**Séance scénarisée**	Filmé : 30 s marche, 30 s course, 10 s sprint, 20 s arrêt.	Accélérations ≥ 2 ; décélérations ≥ 1 ; sprints = 1 ; course ≈ 0:40 (±5 s) ; activité ≈ 44 % (±5 points).	☐
T5.13	**I**	**Contacts**	5 impacts sur sac de plaquage.	Aucune erreur ; noter les pas parasites.	☐
T5.14	**B**	**Endurance logicielle**	`demo_rugby(1200)` en jeu libre.	Aucune erreur ; manques ≈ 0 ; `seance_rugby_NNNN.json` écrit.	☐
T5.15	**B**	**Fichiers et traçabilité**	Après T5.14 : ouvrir le résumé JSON et le CSV d'état.	Fichiers numérotés (la séance précédente n'est pas écrasée) ; `partiel` vaut False ; `erreurs_lecture` et `echantillons_manques` faibles ; le CSV a une ligne par seconde et la colonne vitesse suit bien le déroulé de la séance. *Si échec : journal inactif : lire son champ « erreur » dans le résumé.*	☐

ID	Prio	Test	Procédure	Critère de réussite	OK
T5.16	I	**Sauvegarde périodique**	Lancer `demo_rugby(600)` et couper l'alimentation vers 2 min.	Au rebranchement, le résumé existe avec `partiel=True` et une durée d'environ 90 à 120 s.	☐

Niveau 6 — Mode foot — capteur au mollet

Objectif : valider la charge du processeur, les foulées, la course et les frappes, sans GPS.

Lieu : terrain | **Matériel :** protège-tibia ou bande serrée, ballons, radar, décamètre, chronomètre, vidéo | **Prérequis :** niveaux 2 et 4

> Lancer : `import mode_foot`, puis `mode_foot.demo_foot(durée_s, longueur_levier_m=L)` ou `mode_foot.demo_etalonnage(allure, distance_m)`. Fixer le boîtier sur la face avant/interne du tibia, solidaire de l'os, toujours au même endroit.

ID	Prio	Test	Procédure	Critère de réussite	OK
T6.1	B	**Démarrage**	`demo_foot(30)`, jambe immobile au départ.	Fréquence max ≥ 180 Hz ; biais affiché ; « mesurable jusqu'a 62,8 km/h » pour L = 0,50 m.	☐
T6.2	B	**Charge du processeur**	`demo_foot(120)` en courant et en frappant.	Échantillons manqués < 1 % du total. *Si échec : réduire FREQUENCE_FOOT_HZ (ex. 100) ; refaire T2.9.*	☐
T6.3	B	**Foulées en marche**	Marcher 40 pas.	nb_foulees = 20 ± 1 (1 foulée = 2 pas). *Si échec : ajuster seuil_min / fraction_seuil de creer_detecteur_foulees().*	☐
T6.4	B	**Foulées en course**	Courir 60 pas.	nb_foulees = 30 ± 2.	☐
T6.5	B	**Étalonnage**	Comme T5.8, avec `mode_foot.demo_etalonnage`.	Trois K entre 0,05 et 1,0 sans butée ; modele_foot.json créé.	☐
T6.6	B	**Distance et vitesse**	Refaire T5.9 et T5.10 en mode foot.	Mêmes critères (±5 %).	☐
T6.7	B	**Comptage des frappes**	20 frappes variées (tirs, centres, dégagements), filmées.	tirs = 20 ± 1. *Si échec : frappes manquées : baisser omega_min_dps de DetecteurFrappe.*	☐
T6.8	B	**Faux positifs en sprint**	5 sprints de 30 m sans ballon.	tirs = 0. *Si échec : monter omega_min_dps ; enregistrer frappes et sprints (demo_enregistrement) pour choisir le seuil.*	☐
T6.9	I	**Passes courtes**	10 passes à 10 m.	Décider si elles doivent compter comme des frappes, puis régler seuil_debut_dps / omega_min_dps.	☐
T6.10	I	**Longueur L**	Mesurer sur le joueur : genou (interligne) → zone de contact du pied.	L notée et passée en paramètre longueur_levier_m.	☐
T6.11	B	**Vitesse de frappe**	10 frappes mesurées au radar (vitesse du ballon).	Rapport v_ballon / v_pied stable à ±10 % ; reporter sa moyenne dans `estimer_vitesse_ballon_kmh(…, rapport)`. *Si échec : rapport très dispersé : vérifier la fixation (T6.1) et la valeur de L.*	☐
T6.12	I	**Saturation**	Les frappes les plus fortes du joueur.	« ≥ » affiché quand ω ≥ ~1960 °/s ; noter le % de frappes saturées.	☐
T6.13	I	**Frappe en course**	Course avec frappe sans s'arrêter.	tirs +1 ; la distance n'augmente pas anormalement (nb_foulees_ecartees peut augmenter de 1).	☐

ID	Prio	Test	Procédure	Critère de réussite	OK
T6.14	**B**	**Endurance logicielle**	`demo_foot(1200)` en jeu libre.	Aucune erreur ; `seance_foot_NNNN.json` écrit.	☐
T6.15	**B**	**Journal des frappes**	Après T6.7 : récupérer `frappes_NNNN.bin`, le convertir avec `journal.convertir_csv` et tracer gx, gy, gz.	Un événement par frappe validée ; le pic de |ω| est visible et correspond à la valeur annoncée pendant la séance. *Si échec : 0 événement : vérifier que journal_frappes vaut True et que des frappes ont été validées.*	☐
T6.16	**B**	**Réglage des seuils hors terrain**	À partir des CSV de T6.15 et de sprints enregistrés, comparer les pics de |ω| des frappes et des foulées de sprint.	Un seuil omega_min_dps sépare les deux populations ; le noter et le reporter dans `DetecteurFrappe`. *Si échec : pas de séparation nette : ajouter le critère acc_min_g.*	☐

Niveau 7 — Assistance GPS

Objectif : valider le GPS en secours, le retour à la centrale, la perte de signal, l'étalonnage automatique et la position.

Lieu : terrain, ciel dégagé | **Matériel :** vélo ou chariot, tournevis pour débrancher un fil | **Prérequis :** niveaux 3, 5 et 6

> Faire T7.1 à T7.9 en mode rugby (`demo_rugby(durée_s, avec_gps=True)`), puis T7.10 en mode foot. Par défaut `priorite="imu"` : la centrale donne la vitesse, le GPS sert de secours.

ID	Prio	Test	Procédure	Critère de réussite	OK
T7.1	**B**	**Démarrage avec GPS**	`demo_rugby(300, avec_gps=True)`	Configuration : toutes valeurs True ; gps_ok passe à True ; nb_satellites ≥ 5.	☐
T7.2	**B**	**Priorité à la centrale**	Course régulière.	source « imu » la plupart du temps.	☐
T7.3	**B**	**Secours GPS**	Déplacer le capteur sans faire de pas (porté à vélo ou sur un chariot) à plus de 6 km/h.	source « gps (secours) » ; vitesse ≈ vitesse réelle ; la distance augmente.	☐
T7.4	**B**	**Retour à la centrale**	Reprendre la course après T7.3.	source revient à « imu ».	☐
T7.5	**B**	**Perte du signal**	En course, débrancher 30 s le fil TX du GPS, puis le rebrancher.	gps_ok passe à False en ≤ 1,5 s, source « imu », aucune erreur ; puis gps_ok revient à True.	☐
T7.6	**I**	**Étalonnage automatique**	Supprimer le modèle (`import os; os.remove("modele_rugby.json")`), courir 5 min régulièrement avec GPS, puis refaire T5.9 sans GPS.	K course modifié dans le résumé (clé « modele ») et enregistré ; erreur de T5.9 réduite.	☐
T7.7	**I**	**Position et cap**	Courir un rectangle autour du terrain.	x_m (est) et y_m (nord) suivent le tracé ; cap_deg cohérent avec la direction.	☐
T7.8	**I**	**Arrêt avec GPS**	Rester immobile 2 min.	La distance augmente de moins de 2 m.	☐
T7.9	**I**	**Comparer les priorités**	Un tour de piste de 400 m avec `priorite="imu"`, puis avec `priorite="gps"`.	Noter les deux distances ; garder la priorité la plus juste.	☐
T7.10	**B**	**Mode foot avec GPS**	Refaire T7.1 à T7.5 avec `demo_foot(durée_s, avec_gps=True)`.	Mêmes critères ; la vitesse de frappe n'est pas modifiée par le GPS.	☐

Niveau 8 — Validation du cahier des charges

Objectif : comparer chaque indicateur à une mesure de référence indépendante.

Lieu : terrain | **Matériel :** piste 400 m, radar, chronométrage, vidéo | **Prérequis :** niveaux 5, 6 et 7

Indicateurs de performance

ID	Indicateur	Référence	Protocole	Critère proposé	OK
T8.1	Tirs	Comptage vidéo	Séance de 30 min	Écart ≤ 5 %	☐
T8.2	Tir max	Radar (ballon)	10 frappes fortes	±10 % (hors frappes saturées)	☐
T8.3	Distance	Piste 400 m (couloir 1)	2 tours en course, 1 en marche	±3 % avec GPS, ±5 % sans	☐
T8.4	Activité	Vidéo chronométrée	Séance scénarisée (T5.12)	±5 points	☐
T8.5	Sprint max	Chrono sur 10 m lancés (cellules ou vidéo rapide)	3 sprints	±5 %	☐
T8.6	Course (temps)	Vidéo	Séance scénarisée	±5 %	☐
T8.7	Sprints	Annotation vidéo	10 min de jeu	±1	☐
T8.8	Accélérations / décélérations	Annotation vidéo, ou priorité GPS	10 min de jeu	Même ordre de grandeur ; fixer la définition (seuil 2 m/s²)	☐
T8.9	Position	Lignes du terrain	Rectangle (T7.7)	< 5 m avec GPS	☐

Exigences non fonctionnelles

ID	Exigence	Protocole	Critère	OK
T8.10	Match complet (B)	`demo_foot(5400)` et `demo_rugby(5400)` avec GPS	Aucune erreur, manques < 1 %, résumé JSON écrit	☐
T8.11	FC2 Autonomie	Mesurer le courant (testeur USB) en mode foot à 500 Hz avec GPS ; capacité = courant × durée visée ; puis décharge complète réelle	Durée visée atteinte. À trancher : 10 h (facteur dimensionnant) ou 20 h (FC2 de C1)	☐
T8.12	FC1 Ne pas gêner	10 min de jeu ; questionnaire au joueur	Le boîtier n'a pas bougé ; aucune gêne signalée	☐
T8.13	FC3 Chocs, sueur, intempéries	Chute de 1 m, arrosage, séance sous la pluie	Fonctionnement normal ensuite	☐

Hors du périmètre du code actuel : la base fixe C0 (FP3 repère fixe, FP4 restitution, FP5 affichage temps réel) et la liaison radio entre C1 et C0 ne sont pas encore programmées. Les méthodes `etat()` et `resume()` fournissent déjà les données à transmettre ; ces fonctions seront à tester quand la liaison existera.

Fiche de résultats (niveau 8)

Indicateur	Valeur de référence	Valeur du capteur	Écart (%)	Mode / GPS	OK
Tirs					☐
Tir max (km/h)					☐
Distance (km)					☐
Activité (%)					☐
Sprint max (km/h)					☐
Course (min:s)					☐
Sprints					☐
Accélérations					☐
Décélérations					☐
Position (m)					☐